

Contents

1. What CropLabel does
2. The idea: label two plants, get the whole folder
3. Requirements & installation
4. First-time setup
5. The labeling workflow, step by step
6. All tools reference
7. The "Find" box — telling the labeler what to look for
8. The crop-vs-weed classifier
9. Batch auto-labeling a whole folder
10. Output files
11. Settings reference
12. Troubleshooting

1. What CropLabel does

CropLabel is a browser-based tool for labeling **crop plants, weeds and soil** in field plot images (UAV tiles, plot photos). It produces both:

- **Semantic segmentation** — every pixel classed as soil, crop or weed;
- **Instance segmentation** — every individual plant separated with its own ID, class, score and bounding box.

Each image contains **one crop species** (barley, wheat, canola, oats, ..., selected per image) plus possible weeds. Soil is labeled **completely automatically** — you never touch it.

Semantic value	Class	Color
0	unlabeled (vegetation not yet assigned)	—
1	soil (automatic — includes stubble/straw)	brown
2	crop	green
3	weed	red

2. The idea: label two plants, get the whole folder

The tool is built around a feedback loop between you and two AI systems:

1. **Soil is free.** Opening an image instantly separates living vegetation from everything else (Excess-Green index), so soil, stubble and plot markers are already labeled.
2. **One click labels all similar plants.** Click a single example plant with the ★ tool — SAM3.1 finds every similar-looking plant in the image and labels each one as a separate instance. Do the same once for a weed with the Weed class selected.
3. **Your corrections train a classifier.** Every time you press Save, a crop-vs-weed classifier retrain on all your labeled instances (color, shape and texture features).
4. **The folder labels itself.** " ⚡ Label ALL remaining" then runs on every image: automatic soil + SAM3.1 plant detection + the classifier deciding crop-vs-weed per plant. Review the results, fix the few mistakes, save — each save makes the next batch better.

Measured on barley trial tiles (1600×1600): ~40 plant instances found per image in under half a second on the GPU; a 38-image folder batch-labeled in about 2 minutes.

3. Requirements & installation

- Windows 10/11 64-bit, **NVIDIA GPU** (8 GB+ VRAM recommended) with a current driver
- Python 3.13 64-bit
- A modern browser. The app runs locally at `http://127.0.0.1:8323` — no internet needed after installation, no image ever leaves the computer.

1. Clone the repository and install dependencies:

```
git clone https://github.com/appndpird/CropLabel.git
cd CropLabel
pip install fastapi uvicorn opencv-python numpy pillow tokenizers
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128
pip install git+https://github.com/facebookresearch/sam3.git
pip install einops triton-windows pycocotools timm
pip install "numpy>=2.1" --upgrade # sam3 pins numpy<2 - undo that
```

2. Download the SAM3.1 checkpoint `sam3.1_multiplex.pt` from `huggingface.co/strangervisionhf/sam3.1-st-bf16` (open access, no account needed) and place it in the repo's `models\` folder.
3. Start with `CropLabel.bat` (or `python run_app.py`). The browser opens automatically; keep the console window open while working.

Important: torchvision must be a CUDA build (`+cu128`). A CPU-only torchvision crashes the SAM3.1 engine with a `roi_align` error — reinstall with the `--index-url .../cu128` shown above.

4. First-time setup

1. Click  **Setup** in the header.
2. **Dataset folder** —  Browse to the folder holding the images. Images are found *recursively*, so plot subfolders work (e.g. `tiles\plot_10001\...\png`). Supported: png, jpg, tif, bmp.
3. **Output folder** — where the label files are written (the input folder structure is mirrored there).
4. Leave the other settings at their defaults to start (section 11 explains each one). Press **Save settings** — the image list appears in the left sidebar, grouped by plot/folder.

5. The labeling workflow, step by step

1. **Pick an image** in the sidebar. Dots: grey = pending, blue = auto-labeled (review), green = reviewed, orange = skipped. The image opens with **soil already labeled**.
2. **Set the crop type** once per image (dropdown in the toolbar: barley / wheat / canola / oats / other). It is stored with the labels.
3. **Label the crop:** with the **Crop** class selected and the ★ tool active, click one clear example plant. All similar plants get labeled as individual instances (white outlines show the separation). Add another example click if some were missed; **Shift** + click anything wrongly labeled as a negative example — the search re-runs refined.
4. **Label weeds** (if any): select the **Weed** class, ★ click one weed. Or use the 🖨️ Find box: type what the weed looks like (e.g. *broad leaf plant*) and press Enter.
5. **Fix the details:** 🗑️ deletes a wrong instance, ↔ flips an instance between crop and weed, + adds a single missed plant, 🖌️/🗑️ paint/erase pixels manually. **Ctrl** + **Z** undoes anything.
6. **Save** (or **Ctrl** + **S**) — writes all label files and retrains the crop-vs-weed classifier with your instances. Switching image auto-saves, so nothing is lost.
7. After 2–3 reviewed images, press ⚡ **Label ALL remaining** and let the folder label itself, then review the blue images.

6. All tools reference

Tool	Key	What it does
★	1	Find similar — click ONE example plant (or drag a box around it): SAM3.1 labels every similar plant in the image as separate instances of the selected class. Shift + click = negative example (something it must NOT label). Examples accumulate and refine each other; ★ ✕ clears them.
+	2	Add one — labels just the single plant under the click as one instance (no search).
🗑️	3	Delete — removes the instance under the click (pixels return to soil/unlabeled).
↔	4	Reclassify — the clicked instance becomes the selected class (crop ↔ weed).
🖌️	5	Brush — paint the selected class by hand (semantic only; size slider in the toolbar).
🗑️	6	Eraser — paint back to soil; also removes instance ownership of the erased pixels.

Other controls: mouse wheel = zoom · right-drag or **Space** + drag = pan · **c** / **w** = select Crop/Weed class · **Ctrl** + **Z** / **Y** undo/redo · **Ctrl** + **S** save · **Overlay** slider fades the label view ·

🌟 **Auto-label** runs the full automatic pipeline on the current image · **Skip** marks an image to ignore.

7. The 🤖 "Find" box — telling the labeler what to look for

The text box above the canvas sends any free-text description to SAM3.1 as a concept prompt and labels the matches as the **currently selected class**:

- Select **Weed**, type *broad leaf plant* → broadleaf weeds get labeled.
- Select **Crop**, type *thin grass seedling* → grass-like seedlings get labeled.

Useful phrasings are visual descriptions: *plant, seedling, green plant, grass, leaf, small plant*. Species names (e.g. "ryegrass") usually work worse than descriptions of shape and color. If a prompt returns too much or too little, adjust the **Threshold** in Setup (lower = more detections) and run it again — every run is undoable.

SAM3.1 finds *plants* extremely well but cannot reliably tell crop from weed by name alone — both are green plants. That judgement comes from your examples: the 🌟 clicks and the classifier (next section).

8. The crop-vs-weed classifier

Every saved image contributes its instances (with your corrected classes) as training data. Once at least **3 crop and 3 weed instances** exist, the classifier activates (header pill turns green) and from then on every auto-label and batch run classifies each detected plant as crop or weed automatically, using color, shape and texture features.

- Until then, all detected plants default to **crop** (the safe assumption in a crop trial) — reclassify the weeds you see with 🗑️ and save.
- The classifier improves with every save; there is nothing to configure.
- The header pill shows its state, e.g. `classifier: 74 instances`.

9. Batch auto-labeling a whole folder

⚡ **Label ALL remaining** processes every pending image in the background: automatic soil → SAM3.1 plant instances (using the plant prompts from Setup) → crop/weed classification. Progress shows in the sidebar; you can keep labeling while it runs, and Stop it anytime.

- Results are saved with status **auto** (blue dot) — every one stays fully editable. Open, correct, Save to promote to reviewed (green).
- Speed: roughly 1–3 seconds per image on a GPU.
- Re-running the batch only touches images that are still pending.

Best practice: review 3–5 images first (including at least a few weed examples if the field has weeds), then batch. The classifier then labels crop vs weed for the whole folder instead of defaulting to crop.

10. Output files

Written to the output folder, mirroring the dataset's folder structure. **The semantic mask has the same filename and pixel size as the input image**, so downstream training pipelines can pair image ↔ label by name.

File	Content
<name>.png	Semantic index mask (0/1/2/3), same name & size as the input image.
<name>_instances.png	16-bit instance-ID map (0 = no instance) — the instance segmentation. IDs match the JSON records.
<name>_color.png	Color-coded semantic mask.
<name>_overlay.png	Labels blended over the image for quick visual checks.
<name>.json	Crop type, status, engine, per-class pixel percentages, and one record per instance: id, class, score, bounding box, area (all at original resolution).
records.json	Queue status store (drives the sidebar colors).

11. Settings reference (⚙️ Setup)

Setting	Default	Meaning
Plant prompts	plant, seedling	SAM3.1 text prompts used by 🌟 auto-label and the batch. Comma-separated; results are merged and de-duplicated.
Threshold	0.4	SAM3.1 confidence cut-off. Lower finds more (and more false) plants.
ExG	0.06	Excess-Green vegetation threshold. Lower catches paler/yellowish plants as vegetation; higher pushes more into soil. Also clips every SAM instance so masks never bleed into soil.
Min px	30	Instances smaller than this many pixels (at working resolution) are dropped as noise.
work_res	1024	Internal working resolution (settings.json). Labels are always saved at the original image size.

12. Troubleshooting

Problem	Fix
Header pill: <i>SAM3.1 NOT available</i>	No NVIDIA GPU/driver, or <code>models\sam3.1_multiplex.pt</code> missing, or torchvision is a CPU build (reinstall with <code>--index-url .../cu128</code>). Hover the pill for the exact error.
🌟 finds only a few plants	Click a <i>typical, clearly visible</i> plant; add a second example click. Dragging a snug box around one plant often beats a click. Lower the Threshold for more matches.
Pale/yellow plants counted as soil	Lower the ExG setting (e.g. 0.03) and reopen the image.
Stubble/straw labeled as plants	Raise ExG slightly, raise Min px, or <code>Shift</code> + click a piece of stubble as a 🌟 negative example.
Everything labeled crop, weeds missed	Expected before the classifier has weed examples. Reclassify (🔄) a few weeds, Save, re-run auto-label / batch.
Two touching plants merged into one instance	Delete the merged instance (🗑️), then <code>+</code> click each plant separately, or use a snug 🌟 box on one of them.
App won't start / port in use	Only one instance at a time — port 8323. Close the other console window.
Wrong folder scanned	⚙️ Setup → check both folders → Save settings. The sidebar reloads immediately.

